

TRPO

LAMDA-5

May 27, 2026

我们这次的目标是实现算法 TRPO (**Trust Region Policy Optimization**, <https://arxiv.org/pdf/1502.05477>), 是 2015 年由当时在 UC Berkeley 读博的 John Schulman 及其导师 Pieter Abbeel 等人所提出, 发表在 JMLR, 目前引用量已过万。它成功地将传统运筹学中的“信赖域”方法引入深度强化学习, 首次在深度神经网络模型上为策略优化提供了“单调改进”的数学保证, 这意味着原则上每次更新都会让策略变得更好, 不会出现灾难性退步。

1 代码补全

该项目的实现参考了<https://spinningup.openai.com/en/latest/algorithms/trpo.html>, 请补全文件 `algorithm/trpo.py` 中的 **TODO** 部分 (完善函数 `compute_advantages_from_traj`, `compute_advantages_from_rollout`, 以及 `_update_policy`, 相关功能要求已在注释中标出)。并回答下述问题:

1.1 `collect_trajectories` 实现部分

1. `collect_trajectories` 与普通 rollout 采样方式相比, 数据收集的停止条件分别是什么?

回答: 普通 rollout (`interact_with_envs`) 按固定步数截断: 每个 epoch 每个并行环境交互满 `interact_per_epoch` 步即返回, 不等 episode 结束。

`collect_trajectories` 按轨迹条数截断: 外层循环条件为 `while not self.traj_rollout.full`, 直到存满 `trajnum` 条轨迹才停止; 每条轨迹以“环境终止”或“达到 `max_episode_length`”为内部结束条件。

2. 在 trajectory 采样模式下, 每一条轨迹需要保存哪些信息? 这些信息分别在后续 TRPO 更新中起什么作用?, 当环境返回终止信号时, 轨迹又会如何结束? 未达到最大长度的部分又应如何处理?

回答: 每条轨迹保存 `states`、`actions`、`rewards`、`dones`、`log_probs`, 对应 `add_traj()` 的输入字段:

- `states`: 供 actor 重算 $\log \pi_{\theta}(a|s)$, 供 critic 估计 $V(s_t)$ 。
- `actions`: 与 `states` 配合计算重要性采样比率。
- `rewards`: 从后往前递推 GAE 和 return。
- `dones`: 递推时乘以 $(1 - d_t)$ 切断 episode 边界的 bootstrap。
- `log_probs`: 旧策略概率, 计算 $\exp(\log \pi_{\theta} - \log \pi_{\theta_{old}})$ 。

环境返回终止信号时立即调用 `add_traj()` 保存当前轨迹并重置该 env 的缓冲区。未填满 `max_episode_length` 的剩余位置保持零初始化, `masks` 为 0, 后续用 `masks==1` 过滤掉 padding。

3. `TrajectoryRollout` 中的 `masks` 变量有什么作用? 如果不使用 `masks`, 计算 return 或 advantage 时可能出现什么问题?

回答: `masks[i,t]=1` 表示该位置是真实 transition, `=0` 表示 padding。 `TrajectoryRollout` 预分配形状为 `(trajnum, max_episode_length)`, 短轨迹后必有零填充, `add_traj()` 中 `self.masks[pos,:traj_len]=1` 记录真实长度。

若不使用 `masks`, padding 的零奖励和零状态会污染 return/advantage 的均值与方差, advantage normalization 随之偏移; GAE 递推也可能把无效步当成真实 transition, 导致 critic 和 actor 的梯度方向失真。

4. `collect_trajectories` 在收集完一批轨迹后为什么需要重新初始化环境状态？

回答：当缓冲区满时，某些环境可能停在 episode 中途，其 `observations` 是非起点状态。若不重置，下一轮轨迹从这些中间状态开始，破坏“从初始分布出发”的假设，轨迹边界不干净。`initialize()` 确保每轮采样从 episode 起点开始，使 on-policy 的“采样—更新—清空”流程清晰，避免上一批残留状态污染下一批新策略的数据。

1.2 `_update_policy` 实现部分

该部分理论主要依赖于 **Trust Region Policy Optimization** 中的 **Theorem 1**，代码实现参考 <https://spinningup.openai.com/en/latest/algorithms/trpo.html> 中的 **Algorithm 1**：

1. 解释该公式中每一项的实际意义，这个公式在什么情况下可以保证策略的单调改进？由此直观解释 TRPO 算法的设计思路（优化目标和约束的设计？先概述你的大致想法，具体细节在后续提问中体现）。

回答：TRPO 的 Theorem 1：

$$\eta(\pi_{\text{new}}) \geq L_{\pi_{\text{old}}}(\pi_{\text{new}}) - C D_{\text{TV}}^{\max}(\pi_{\text{old}}, \pi_{\text{new}})^2.$$

η 是新策略真实期望回报； $L_{\pi_{\text{old}}}$ 是固定旧策略状态分布下的代理目标（重要性采样加权 advantage）；后项是新旧策略 TV 距离的惩罚，量化用旧分布近似新分布的误差。只要代理目标提升量超过惩罚项，下界上升，策略性能单调改进。由此 TRPO 的设计思路是：最大化 L （鼓励 advantage 为正的動作）同时约束策略变化幅度，即在“可信域”内找最优更新。

2. 和文章 *Approximately optimal approximate reinforcement learning* 中的 Theorem 4.1（即 TRPO 论文中的公式 (6)）作对比，TRPO 作者在分析上的处理技术有什么具体的不同，使得 TRPO 可以推广到所有一般的随机策略类而不是给定的混合策略类？

回答：Kakade-Langford 要求新策略为混合形式 $\pi_{\text{new}} = (1 - \alpha)\pi_{\text{old}} + \alpha\pi'$ ，偏离旧策略的概率显式等于 α ，分析局限于此混合策略类。TRPO 令

$$\alpha = \max_s D_{\text{TV}}(\pi_{\text{old}}(\cdot|s), \pi_{\text{new}}(\cdot|s))$$

作为“单步采到不同动作的概率上界”，不要求 π_{new} 有特定结构，从而推广到任意随机策略（包括神经网络参数化的高斯策略）。

3. TRPO 中策略更新的 surrogate objective $L_{\pi_{\text{old}}}(\pi_{\text{new}})$ 一项在实际算法中是如何设计的？

回答：用旧策略采样数据做 Monte Carlo 估计，重要性采样修正采样偏差：

$$\hat{L}(\theta) = \frac{1}{N} \sum_{t=1}^N \frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)} \hat{A}_t.$$

对应 `trpo.py`：先保存采样时的 `old_log_probs`，更新时用 `self.agent.log_prob(states, actions)` 重算新策略概率，再令 `ratio = torch.exp(new_log_probs - old_log_probs)`，最后 `surrogate = (ratio * advantages).mean()` 即为 actor 最大化的目标。

4. 为什么 TRPO 需要约束新旧策略之间的 KL divergence 而不是定理中的 TV 距离？在实际算法中我们采用 KL 散度的二阶近似 $D_{\text{KL}}(\pi_{\theta_{\text{old}}} \| \pi_{\theta}) \sim \frac{1}{2} \Delta \theta^T (\nabla_{\theta}^2 D_{\text{KL}}) \Delta \theta$ ，延续该思路解释为什么在 Algorithm 1 中我们会采用 line search 搜索参数 (Algorithm 1 line 8&9)？

回答：KL 散度对高斯策略有解析式且可微，TV 距离不具备此优势；Pinsker 不等式保证 KL 控制 TV，故用 KL 替代。KL 的二阶近似 $\frac{1}{2} \Delta \theta^T H \Delta \theta \leq \delta$ (H 为 Fisher 信息矩阵) 将约束转化为线性方程 $Hx = g$ ，`trpo.py` 用共轭梯度求解得自然梯度方向，再由 `stepsize = sqrt(2*delta / xHx)` 算出理论步长。

但二阶近似在非线性网络下仅局部有效，实际 KL 可能超过 δ 而 surrogate 也可能不提升，因此需要 line search：沿自然梯度方向指数衰减步长逐步回退，找到同时满足 `ls_surrogate > surrogate` 且 `ls_kl <= self.delta` 的最大步长（即 `candidate_params = old_params + alpha * stepsize * gradient_direction`）；若全部步长失败则恢复旧参数。

2 探索部分

请从以下五个环境中至少选择三个进行测试：**Humanoid-v5**, **Ant-v5**, **HalfCheetah-v5**, **Hopper-v5**, **Walker2d-v5** 并思考下述问题：

1. 在相同超参数设置下，三个环境的训练难度是否一致？请结合 reward 曲线进行分析。
2. 哪个环境中 trajectory 与 rollout 的样本利用率差异最明显？可能的原因是什么？
3. 在 trajectory 模式和 rollout 模式下，advantage 与 return 的计算方式有什么不同？

2.1 Advantage Normalization 的影响

1. 请分别设置 `advan_norm=true` 和 `advan_norm=false` 进行实验。两组实验的平均回报曲线有何差异？
2. 加入 advantage normalization 后，训练过程是否更加稳定？请结合 reward 曲线、KL divergence 或 surrogate objective 的变化进行分析。
3. 在你选择的测试环境中，advantage normalization 是否总是带来性能提升？如果不是，可能的原因是什么？

2.2 Trajectory 与 Rollout 的样本利用率比较

1. 在 rollout 模式下，每次策略更新实际使用了多少个 transition？在 trajectory 模式下，每次策略更新又实际使用了多少个有效 transition？
2. 对比 trajectory 与 rollout 两种模式，哪一种样本利用率更高？
3. 样本利用率更高是否一定意味着训练效果更好？请结合实验曲线进行分析。

3 总结与反思

1. 在本项目中，`collect_trajectories` 的实现是否会改变 TRPO 的 on-policy 特性？说出你的判断理由。
2. 如果只能保留一种采样方式，你会选择 trajectory 还是 rollout？请结合样本利用率和训练性能说明理由。
3. 完成实验报告的撰写后，你对 TRPO 算法是否有了新的感悟或认识？(这个问题不建议 AI 辅助生成，AI 时代最珍贵的是人类自我的想法。相信你写完报告后总会有一些自己的思考，或许是疑惑或许是感悟，完善与否不重要，写出你自己的想法就好。)
4. 针对本次算法实现，你是否有什么想法和想改进的建议，你在实现过程中有没有因为项目 README.md 或者实验报告引导和介绍问题导致卡了很久，如果有，请详细说明，并给出你的建议。关于代码/实验报告问题难度你是否有感觉很难/简单？